SE3082

AWS EC2 Lab - Screenshots and Calculations

Lab 08

Exercise 7: Monte Carlo Pi with Buffered Send

IT24610328

On instance with 2 vCPUs (t3.micro)

Nano - creating the Monte Carlo Pi code with Buffered Send

```
  GNU nano 6.2                                pi_bsend.cc
#include <cstdio>
#include <cstdlib>
#include <mpi.h>
int main(void)
{
    int rank, size;
    MPI_Status status;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // IT24610328 - Ex3 - Added total points and per process share
    const long long N = 10000000;
    long long mine = N / size;
    if (rank == size - 1) mine += N % size;

    // IT24610328 - Ex3 - Added barrier and timer
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    // IT24610328 - Ex3 - Added Monte Carlo loop with a different seed per rank
    unsigned int seed = 12345 + rank;
    long long hits = 0;
    for (long long i = 0; i < mine; i++) {
        double x = (double)rand_r(&seed) / RAND_MAX;
        double y = (double)rand_r(&seed) / RAND_MAX;
        if (x * x + y * y <= 1.0) hits++;
    }

    // IT24610328 - Ex3 - Added send of hit counts to rank 0 and receive from each rank in order
    // IT24610328 - Ex6 - Changed receive source from src to MPI_ANY_SOURCE
    // IT24610328 - Ex6 - Added print of the rank each count arrived from
    // IT24610328 - Ex7 - Changed MPI_Send to MPI_Bsend
    // IT24610328 - Ex7 - Added buffer attach and detach around the send
    if (rank != 0) {
        int bufsize = sizeof(long long) + MPI_BSEND_OVERHEAD;
        char *buf = (char *)malloc(bufsize);
        MPI_Buffer_attach(buf, bufsize);
        MPI_Bsend(&hits, 1, MPI_LONG_LONG, 0, 0, MPI_COMM_WORLD);
        MPI_Buffer_detach(&buf, &bufsize);
        free(buf);
    } else {
        long long total = hits, other;
        for (int k = 1; k < size; k++) {
            MPI_Recv(&other, 1, MPI_LONG_LONG, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &status);
            printf("Received %lld hits from rank %d\n", other, status.MPI_SOURCE);
            total += other;
        }
        double t1 = MPI_Wtime();
        printf("Processes = %d\n", size);
        printf("Pi = %.6f\n", 4.0 * total / N);
        printf("Time = %f seconds\n", t1 - t0);
    }
    MPI_Finalize();
}
```

Output from running the Buffered Send version (4 processes)

